

P1028R2: SG14 **status_code** and standard **error** object for P0709 Zero-overhead deterministic exceptions

Document #: P1028R2
Date: 2019-06-10
Project: Programming Language C++
Library Evolution Working Group
SG14 Low Latency study group
Reply-to: Niall Douglas
<s_sourceforge@nedprod.com>

A proposal for the replacement, in new code, of the system header `<system_error>` with a substantially refactored and lighter weight design, which meets modern C++ design and implementation. This paper received the following vote at the May 2018 meeting of SG14: 8/2/1/0/0 (SF/WF/N/WA/SA).

A C++ 11 reference implementation of the proposed replacement can be found at <https://github.com/ned14/status-code>. Support for the proposed objects has been wired into Boost.Outcome [1], a library-only implementation of [P0709]. The proposed objects have received extensive field testing in existing code bases, and have been found to work very well.

The reference implementation has been found to work well on recent editions of GCC, clang and Microsoft Visual Studio, on x86, x64, ARM and AArch64. It has been in production use for a year now, and has been shipping in Boost from v1.70 onwards as part of Outcome.Experimental.

Some of the design patterns used in this proposal were merged into Boost.System, and have been proposed for merging into `<system_error>`:

- [P1195] *Making `<system_error>` constexpr*
- [P1196] *Value-based `std::error_category` comparison*
- [P1197] *A non-allocating overload of `error_category::message()`*
- [P1198] *Adding `error_category::failed()`*

These rub down the roughest corners on `<system_error>`, and little source code breakage has been observed for Boost C++ library users from the changes. I would thus support the merging of the above proposals into the C++ standard.

However, this proposal is a much richer and more powerful framework than `<system_error>`. Indeed, it can almost completely replace the dynamic exception mechanism, and has been proposed as the `std::error` implementation for [P0709] *Zero overhead deterministic exceptions* in [P1095] *Zero overhead deterministic failure*.

Changes since R1:

- Erased status codes, which includes proposed `std::error`, are now always move-only. They *may* be copyable via `.clone()`, if the original payload type is copyable.
- Everything which was keyed around `TriviallyCopyable` is now keyed around `MoveRelocatable`, which is a stand-in for whichever relocatable object proposal gets chosen by WG21.
- Added `is_errored_status_code` trait.
- Added custom status code domain for `getaddrinfo()`.
- Added status code pointers, which are a system code compatible pointer to a dynamically allocated status code. These let you ‘wrap up’ a ‘fat’ status code into a `std::error` compatible status code, with all operations on the ‘thin’ status code indirecting to the ‘fat’ status code.

Contents

1	Introduction	3
2	Impact on the Standard	4
3	Proposed Design	5
3.1	<code>status_code_domain</code>	5
3.2	Traits	8
3.3	<code>status_code<void></code>	9
3.4	<code>detail::status_code_storage<DomainType></code>	10
3.5	<code>status_code<DomainType></code>	11
3.6	<code>status_code<erased<TRIVIALY_COPYABLE_OR_MOVE_RELOCATING_TYPE>></code>	13
3.7	Exception types	14
3.8	Generic error coding	16
3.9	<code>errored_status_code<DomainType></code>	17
3.10	<code>errored_status_code<erased<TRIVIALY_COPYABLE_OR_MOVE_RELOCATING_TYPE>></code>	19
3.11	OS specific codes, and erased system code	21
3.12	Proposed <code>std::error</code> object	22
3.13	iostream printing support	22
3.14	<code>status_code_ptr</code>	22
4	Design decisions, guidelines and rationale	23
4.1	Do not cause <code>#include <string></code>	23
4.2	All constexpr sourcing, construction and destruction	24
4.3	Header only libraries can now safely define custom code categories	24
4.4	No more <code>if(!ec)</code>	25
4.5	No more filtering codes returned by system APIs	26
4.6	All comparisons between codes are now semantic, not literal	26
4.7	<code>std::error_condition</code> is removed entirely	26
4.8	<code>status_code</code> ’s value type is set by its domain	27
4.9	<code>status_code<DomainType></code> is type erasable	27

4.10	More than one ‘system’ error coding domain: <code>system_code</code>	27
4.11	<code>std::errc</code> gets its own code domain <code>generic_code</code> , eliminating <code>std::error_condition</code>	28
5	Technical specifications	28
6	Frequently asked questions	28
6.1	Implied in this design is that code domains must do nothing in their constructor and destructors, as multiple instances are permitted and both must be trivial and constexpr. How then can dynamic per-domain initialisation be performed e.g. setting up at run time a table of localised message strings?	28
7	Acknowledgements	29
8	References	29

1 Introduction

The `<system_error>` header entered the C++ standard in the C++ 11 standard, the idea for which having been split off from the Filesystem TS proposal into its own [N2066] proposal back in 2006. Despite its relative lack of direct usage by the C++ userbase, according to [2], `<system_error>` has become one of the most common internal dependencies for all other standard header files, frequently constituting up to 20% of all the tokens brought into the compiler by other standard header files e.g. `<array>`, `<complex>` or `<optional>`. In this sense, it is amongst the most popular system headers in the C++ standard library.

So why would anyone want to replace it? It unfortunately suffers from a number of design problems only now apparent after twelve years of hindsight, which makes it low hanging fruit in the achievement of the ‘reduce compile time’ and ‘alternatives to complicated and/or error-prone features’ goals listed in [P0939] *Direction for ISO C++*. We, from Study Group 14 (the GameDev & Low Latency WG21 working group), listed many of these problems in [P0824], and after an extensive period of consultation with other stakeholders including the Boost C++ Libraries, we thence designed and implemented an improved substitute which does not have those problems. It is this improved, fully backwards compatible, design that we propose now.

This proposed library may be useful as the standardised implementation of the lightweight throwable `error` object as proposed by [P0709] *Zero-overhead deterministic exceptions: Throwing values*. It is [P0829] *Freestanding C++* compatible i.e. without dependency on any STL or language facility not usable on embedded systems.

An example of use:

```

1 std::system_code sc; // default constructs to empty
2 native_handle_type h = open_file(path, sc);
3 // Is the code a failure?
4 if(sc.failure())
5 {
6     // Do semantic comparison to test if this was a file not found failure
7     // This will match any system-specific error codes meaning a file not found

```

```

8   if(sc != std::errc::no_such_file_or_directory)
9   {
10      std::cerr << "FATAL: " << sc.message().c_str() << std::endl;
11      std::terminate();
12   }
13 }

```

The above is 100% portable code. Meanwhile, the implementation of `open_file()` might be these:

<pre> 1 // POSIX implementation 2 using native_handle_type = int; 3 native_handle_type open_file(const char *path, 4 std::system_code &sc) noexcept 5 { 6 sc.clear(); // clears to empty 7 native_handle_type h = ::open(path, O_RDONLY); 8 if(-1 == h) 9 { 10 // posix_code type erases into system_code 11 sc = std::posix_code(errno); 12 } 13 return h; 14 } </pre>	<pre> 1 // Microsoft Windows implementation 2 using native_handle_type = HANDLE; 3 native_handle_type open_file(const wchar_t *path, 4 std::system_code &sc) noexcept 5 { 6 sc.clear(); // clears to empty 7 native_handle_type h = CreateFile(path, 8 GENERIC_READ, 9 FILE_SHARE_READ FILE_SHARE_WRITE 10 FILE_SHARE_DELETE, 11 nullptr, 12 OPEN_EXISTING, 13 FILE_ATTRIBUTE_NORMAL, 14 nullptr 15); 16 if(INVALID_HANDLE_VALUE == h) 17 { 18 // win32_code type erases into system_code 19 sc = std::win32_code(GetLastError()); 20 } 21 return h; </pre>
---	---

2 Impact on the Standard

The proposed library is a pure-library solution.

There is an optional dependency on a core language enhancement adding support for relocating objects. This could be any one of (or some proposal paper yet to come):

1. P1029 *move = relocates* <https://wg21.link/P1029>.

This proposes a new default for move constructors, `= relocates`. It is used by the programmer to guarantee to the C++ abstract machine that move constructors equal the bit copy of source object's storage over the destination object's storage, followed by bit copy of a constexpr default constructed instance of the object over the source object's storage. By using `= relocates`, the programmer also guarantees that the destructor, when called on a default constructed instance, has no visible side effects.

This is not a generalised solution, but rather solves a specific problem with a specific workaround.

2. P1144 *Object relocation in terms of move plus destroy* <https://wg21.link/P1144>.

This is a more generalised approach than P1029, which relies on adding a new operation to the C++ abstract machine of inseparable move + destroy.

3. P1631 *Object detachment and attachment* <https://wg21.link/P1631>.

This is an even more generalised approach than the preceding two proposals, so much so that it doesn't actually propose relocation 😊. It adds object detachment and attachment operations to the abstract machine, these being a one-way reinterpretation of live object storage into a byte array, and a one-way reinterpretation of a byte array into a live object. One can thus implement relocation by detaching the object, bit copying its detached representation elsewhere, and reattaching the object, *if* that object is both detachable and attachable (not all object types are).

3 Proposed Design

3.1 status_code_domain

```
1  /*! The main workhorse of the system_error2 library, can be typed
2  ('status_code<DomainType>'), erased-immutable ('status_code<void>') or
3  erased-mutable ('status_code<erased<T>>').
4
5  Be careful of placing these into containers! Equality and inequality operators are
6  *semantic* not exact. Therefore two distinct items will test true! To help prevent
7  surprise on this, 'operator<' and 'std::hash<>' are NOT implemented in order to
8  trap potential incorrectness. Define your own custom comparison functions for your
9  container which perform exact comparisons.
10 */
11 template <class DomainType> class status_code;
12
13 class _generic_code_domain;
14
15 /*! The generic code is a status code with the generic code domain, which is that of 'errc' (POSIX).
16 using generic_code = status_code<_generic_code_domain>;
```

```
1  /*! Abstract base class for a coding domain of a status code.
2  */
3  class status_code_domain
4  {
5      template <class DomainType> friend class status_code;
6      template <class StatusCode> friend class indirecting_domain;
7
8  public:
9      /*! Type of the unique id for this domain.
10     using unique_id_type = unsigned long long;
11
12     /*! (Potentially thread safe) Reference to a message string.
13
14     Be aware that you cannot add payload to implementations of this class.
15     You get exactly the 'void *[3]' array to keep state, this is usually
16     sufficient for a 'std::shared_ptr<>' or a 'std::string'.
17
```

```

18 You can install a handler to be called when this object is copied,
19 moved and destructed. This takes the form of a C function pointer.
20 */
21 class string_ref
22 {
23 public:
24     //! The value type
25     using value_type = const char;
26     //! The size type
27     using size_type = size_t;
28     //! The pointer type
29     using pointer = const char*;
30     //! The const pointer type
31     using const_pointer = const char*;
32     //! The iterator type
33     using iterator = const char*;
34     //! The const iterator type
35     using const_iterator = const char*;
36
37 protected:
38     //! The operation occurring
39     enum class _thunk_op
40     {
41         copy,
42         move,
43         destruct
44     };
45     //! The prototype of the handler function. Copies can throw, moves and destructs cannot.
46     using _thunk_spec = void (*)(string_ref *dest, const string_ref *src, _thunk_op op);
47
48     //! Pointers to beginning and end of character range
49     pointer _begin{}, _end{};
50
51     //! Three 'void*' of state
52     void *_state[3]{}; // at least the size of a shared_ptr
53
54     //! Handler for when operations occur
55     const _thunk_spec _thunk{nullptr};
56
57     constexpr explicit string_ref(_thunk_spec thunk) noexcept;
58
59 public:
60     //! Construct from a C string literal
61     constexpr explicit string_ref(const char *str, size_type len = static_cast<size_type>(-1),
62                                   void *state0 = nullptr, void *state1 = nullptr,
63                                   void *state2 = nullptr, _thunk_spec thunk = nullptr) noexcept;
64     //! Copy construct the derived implementation.
65     string_ref(const string_ref &o);
66     //! Move construct the derived implementation.
67     string_ref(string_ref &&o) noexcept;
68     //! Copy assignment
69     string_ref &operator=(const string_ref &o);
70     //! Move assignment
71     string_ref &operator=(string_ref &&o) noexcept;
72     //! Destruction
73     ~string_ref();

```

```

74
75     //! Returns whether the reference is empty or not
76     [[nodiscard]] bool empty() const noexcept;
77     //! Returns the size of the string
78     size_type size() const noexcept;
79     //! Returns a null terminated C string
80     const_pointer c_str() const noexcept;
81     //! Returns a null terminated C string
82     const_pointer data() const noexcept;
83     //! Returns the beginning of the string
84     iterator begin() noexcept;
85     //! Returns the beginning of the string
86     const_iterator begin() const noexcept;
87     //! Returns the beginning of the string
88     const_iterator cbegin() const noexcept;
89     //! Returns the end of the string
90     iterator end() noexcept;
91     //! Returns the end of the string
92     const_iterator end() const noexcept;
93     //! Returns the end of the string
94     const_iterator cend() const noexcept;
95 };
96
97 /*! A reference counted, threadsafe reference to a message string.
98 */
99 class atomic_refcounted_string_ref : public string_ref
100 {
101     struct _allocated_msg
102     {
103         mutable std::atomic<unsigned> count;
104     };
105     _allocated_msg *_msg() noexcept;
106     const _allocated_msg *_msg() const noexcept;
107
108     static void _refcounted_string_thunk(string_ref *_dest, const string_ref *_src, _thunk_op op)
109         noexcept;
110 public:
111     //! Construct from a C string literal allocated using 'malloc()'.
112     explicit atomic_refcounted_string_ref(const char *str, size_type len = static_cast<size_type>(-1),
113         void *state1 = nullptr, void *state2 = nullptr) noexcept;
114 };
115
116 private:
117     unique_id_type _id;
118
119 protected:
120     /*! Use [https://www.random.org/cgi-bin/randbyte?nbytes=8&format=h](https://www.random.org/cgi-bin/
121         randbyte?nbytes=8&format=h) to get a random 64 bit id.
122     Do NOT make up your own value. Do NOT use zero.
123     */
124     constexpr explicit status_code_domain(unique_id_type id) noexcept;
125     //! No public copying at type erased level
126     status_code_domain(const status_code_domain &) = default;
127     //! No public moving at type erased level
128     status_code_domain(status_code_domain &&) = default;

```

```

128  ///! No public assignment at type erased level
129  status_code_domain &operator=(const status_code_domain &) = default;
130  ///! No public assignment at type erased level
131  status_code_domain &operator=(status_code_domain &&) = default;
132  ///! No public destruction at type erased level
133  ~status_code_domain() = default;
134
135 public:
136  ///! True if the unique ids match.
137  constexpr bool operator==(const status_code_domain &o) const noexcept;
138  ///! True if the unique ids do not match.
139  constexpr bool operator!=(const status_code_domain &o) const noexcept;
140  ///! True if this unique is lower than the other's unique id.
141  constexpr bool operator<(const status_code_domain &o) const noexcept;
142
143  ///! Returns the unique id used to identify identical category instances.
144  constexpr unique_id_type id() const noexcept;
145  ///! Name of this category.
146  virtual string_ref name() const noexcept = 0;
147
148 protected:
149  ///! True if code means failure.
150  virtual bool _do_failure(const status_code<void> &code) const noexcept = 0;
151  ///! True if code is (potentially non-transitively) equivalent to another code in another domain.
152  virtual bool _do_equivalent(const status_code<void> &code1, const status_code<void> &code2) const
153      noexcept = 0;
154  ///! Returns the generic code closest to this code, if any.
155  virtual generic_code _generic_code(const status_code<void> &code) const noexcept = 0;
156  ///! Return a reference to a string textually representing a code.
157  virtual string_ref _do_message(const status_code<void> &code) const noexcept = 0;
158  ///! Throw a code as a C++ exception.
159  [[noreturn]] virtual void _do_throw_exception(const status_code<void> &code) const = 0;
160 };

```

3.2 Traits

```

1  ///! Namespace for user injected mixins
2  namespace mixins
3  {
4      template <class Base, class T> struct mixin : public Base
5      {
6          using Base::Base;
7      };
8  }
9
10  /*! A tag for an erased value type for 'status_code<D>'.
11  Available only if 'ErasedType' satisfies 'traits::is_move_relocating<ErasedType>::value'.
12  */
13  template <class ErasedType>
14  requires(traits::is_move_relocating<ErasedType>::value)
15  struct erased
16  {
17      using value_type = ErasedType;
18  };

```



```

19
20 /// Trait returning true if the type is a status code.
21 template <class T> struct is_status_code;
22 template <class T> static constexpr bool is_status_code_v;

```

3.3 status_code<void>

```

1  /*! A type erased lightweight status code reflecting empty, success, or failure.
2  Differs from 'status_code<erased<>>' by being always available irrespective of
3  the domain's value type, but cannot be copied, moved, nor destructed. Thus one
4  always passes this around by const lvalue reference.
5  */
6  template <> class status_code<void>
7  {
8      template <class T> friend class status_code;
9
10 public:
11     ///! The type of the domain.
12     using domain_type = void;
13     ///! The type of the status code.
14     using value_type = void;
15     ///! The type of a reference to a message string.
16     using string_ref = typename status_code_domain::string_ref;
17
18 protected:
19     const status_code_domain *_domain{nullptr};
20
21 protected:
22     ///! No default construction at type erased level
23     status_code() = default;
24     ///! No public copying at type erased level
25     status_code(const status_code &) = default;
26     ///! No public moving at type erased level
27     status_code(status_code &&) = default;
28     ///! No public assignment at type erased level
29     status_code &operator=(const status_code &) = default;
30     ///! No public assignment at type erased level
31     status_code &operator=(status_code &&) = default;
32     ///! No public destruction at type erased level
33     ~status_code() = default;
34
35     ///! Used to construct a non-empty type erased status code
36     constexpr explicit status_code(const status_code_domain *v) noexcept;
37
38 public:
39     ///! Return the status code domain.
40     constexpr const status_code_domain &domain() const noexcept;
41     ///! True if the status code is empty.
42     [[nodiscard]] constexpr bool empty() const noexcept;
43
44     ///! Return a reference to a string textually representing a code.
45     string_ref message() const noexcept;
46     ///! True if code means success.
47     bool success() const noexcept;

```

```

48  ///! True if code means failure.
49  bool failure() const noexcept;
50
51  /*! True if code is strictly (and potentially non-transitively) semantically equivalent to
52  another code in another domain.
53
54  Note that usually non-semantic i.e. pure value comparison is used when the other
55  status code has the same domain. As 'equivalent()' will try mapping to generic code,
56  this usually captures when two codes have the same semantic meaning in 'equivalent()'.
57  */
58  template <class T> bool strictly_equivalent(const status_code<T> &o) const noexcept;
59
60  /*! True if code is equivalent, by any means, to another code in another domain
61  (guaranteed transitive).
62
63  Firstly 'strictly_equivalent()' is run in both directions. If neither succeeds, each domain
64  is asked for the equivalent generic code and those are compared.
65  */
66  template <class T> inline bool equivalent(const status_code<T> &o) const noexcept;
67
68  ///! Throw a code as a C++ exception.
69  [[noreturn]] void throw_exception() const;
70 };

```

3.4 detail::status_code_storage<DomainType>

```

1  namespace detail
2  {
3      template <class DomainType> struct get_domain_value_type
4      {
5          using domain_type = DomainType;
6          using value_type = typename domain_type::value_type;
7      };
8      template <class ErasedType> struct get_domain_value_type<erased<ErasedType>>
9      {
10         using domain_type = status_code_domain;
11         using value_type = ErasedType;
12     };
13     template <class DomainType> class status_code_storage : public status_code<void>
14     {
15     public:
16         ///! The type of the domain.
17         using domain_type = typename get_domain_value_type<DomainType>::domain_type;
18         ///! The type of the status code.
19         using value_type = typename get_domain_value_type<DomainType>::value_type;
20         ///! The type of a reference to a message string.
21         using string_ref = typename domain_type::string_ref;
22
23         ///! Return the status code domain.
24         constexpr const domain_type &domain() const noexcept;
25
26         ///! Reset the code to empty.
27         constexpr void clear() noexcept;
28     };

```

```

29     ///! Return a reference to the 'value_type'.
30     constexpr value_type &value() & noexcept;
31     ///! Return a reference to the 'value_type'.
32     constexpr value_type &&value() && noexcept;
33     ///! Return a reference to the 'value_type'.
34     constexpr const value_type &value() const & noexcept;
35     ///! Return a reference to the 'value_type'.
36     constexpr const value_type &&value() const && noexcept;
37
38 protected:
39     status_code_storage() = default;
40     status_code_storage(const status_code_storage &) = default;
41     status_code_storage(status_code_storage &&) = default;
42     status_code_storage &operator=(const status_code_storage &) = default;
43     status_code_storage &operator=(status_code_storage &&) = default;
44     ~status_code_storage() = default;
45
46     value_type _value{};
47     struct _value_type_constructor { };
48     template <class... Args>
49     constexpr status_code_storage(_value_type_constructor /*unused*/, const status_code_domain *v,
50                                   Args &&... args);
51 } // namespace detail

```

3.5 status_code<DomainType>

```

1  /*! A lightweight, typed, status code reflecting empty, success, or failure.
2  This is the main workhorse of the system_error2 library.
3
4  An ADL discovered helper function 'make_status_code(T, Args...)' is looked up by one
5  of the constructors. If it is found, and it generates a status code compatible with this
6  status code, implicit construction is made available.
7
8  You may mix in custom member functions and member function overrides by injecting a specialisation of
9  'mixins::mixin<Base, YourDomainType>'. Your mixin must inherit from 'Base'.
10 */
11 template <class DomainType>
12 requires(
13     (!std::is_default_constructible<typename DomainType::value_type>::value
14      || std::is_nothrow_default_constructible<typename DomainType::value_type>::value)
15     && (!std::is_move_constructible<typename DomainType::value_type>::value
16        || std::is_nothrow_move_constructible<typename DomainType::value_type>::value)
17     && std::is_nothrow_destructible<typename DomainType::value_type>::value
18 )
19 class status_code : public mixins::mixin<detail::status_code_storage<DomainType>, DomainType>
20 {
21     template <class T> friend class status_code;
22
23 public:
24     ///! The type of the domain.
25     using domain_type = DomainType;
26     ///! The type of the status code.
27     using value_type = typename domain_type::value_type;

```

```

28  ///! The type of a reference to a message string.
29  using string_ref = typename domain_type::string_ref;
30
31 public:
32  ///! Default construction to empty
33  status_code() = default;
34  ///! Copy constructor
35  status_code(const status_code &) = default;
36  ///! Move constructor
37  status_code(status_code &&) = default;
38  ///! Copy assignment
39  status_code &operator=(const status_code &) = default;
40  ///! Move assignment
41  status_code &operator=(status_code &&) = default;
42  ~status_code() = default;
43
44  ///! Return a copy of the code.
45  constexpr status_code clone() const;
46
47  ///! Implicit construction from any type where an ADL discovered
48  ///! 'make_status_code(T, Args ...)' returns a 'status_code'.
49  template <class T, class... Args,
50           class MakeStatusCodeOutType = decltype(make_status_code(std::declval<T>(), std::declval<
51           Args>()...))> // ADL enable
52  requires(!std::is_same<typename std::decay<T>::type, status_code>::value // not copy/move of self
53           && is_status_code<MakeStatusCodeOutType>::value // ADL makes a status code
54           && std::is_constructible<status_code, MakeStatusCodeOutType>::value // ADLed status code is
55           compatible
56           )
57  constexpr status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(std::declval<T>(),
58           std::declval<Args>()...)));
59
60  ///! Explicit in-place construction.
61  template <class... Args>
62  constexpr explicit status_code(in_place_t /*unused */, Args &&... args) noexcept(std::
63           is_nothrow_constructible<value_type, Args &&...>::value);
64
65  ///! Explicit in-place construction from initialiser list.
66  template <class T, class... Args>
67  constexpr explicit status_code(in_place_t /*unused */, std::initializer_list<T> il, Args &&... args)
68           noexcept(std::is_nothrow_constructible<value_type, std::initializer_list<T>, Args &&...>::
69           value);
70
71  ///! Explicit copy construction from a 'value_type'.
72  constexpr explicit status_code(const value_type &v) noexcept(std::is_nothrow_copy_constructible<
73           value_type>::value);
74
75  ///! Explicit move construction from a 'value_type'.
76  constexpr explicit status_code(value_type &&v) noexcept(std::is_nothrow_move_constructible<
77           value_type>::value);
78
79  /*! Explicit construction from an erased status code. Available only if
80  'value_type' is trivially destructible and 'sizeof(status_code) <= sizeof(status_code<erased<>>)'.
81  Does not check if domains are equal.
82  */
83  template <class ErasedType>

```

```

76 requires(detail::type_erasure_is_safe<ErasedType, value_type>::value)
77 constexpr explicit status_code(const status_code<erased<ErasedType>> &v) noexcept(std::
    is_nothrow_copy_constructible<value_type>::value);
78
79 //!< Assignment from a 'value_type'.
80 constexpr status_code &operator=(const value_type &v) noexcept(std::is_nothrow_copy_assignable<
    value_type>::value);
81
82 //!< Return a reference to a string textually representing a code.
83 string_ref message() const noexcept;
84 };

```

3.6 status_code<erased<TRIVIALY_COPYABLE_OR_MOVE_RELOCATING_TYPE>>

```

1  /*! Type erased, move-only status_code, unlike 'status_code<void>' which cannot be moved nor
2  destroyed. Available only if 'erased<>' is available, which is when the domain's type is trivially
3  copyable or is move relocatable, and if the size of the domain's typed error code is less than
4  or equal to this erased error code. Copy construction is disabled, but if you want a copy call
5  '.clone()'.
6
7  An ADL discovered helper function 'make_status_code(T, Args...)' is looked up by one of the
8  constructors. If it is found, and it generates a status code compatible with this status code,
9  implicit construction is made available.
10 */
11 template <class ErasedType> class status_code<erased<ErasedType>>
12 : public mixins::mixin<detail::status_code_storage<erased<ErasedType>>, erased<ErasedType>>
13 {
14     template <class T> friend class status_code;
15
16 public:
17     //!< The type of the domain (void, as it is erased).
18     using domain_type = void;
19     //!< The type of the erased status code.
20     using value_type = ErasedType;
21     //!< The type of a reference to a message string.
22     using string_ref = typename _status_code<void>::string_ref;
23
24 public:
25     //!< Default construction to empty
26     status_code() = default;
27     //!< Copy constructor
28     status_code(const status_code &) = delete;
29     //!< Move constructor
30     status_code(status_code &&) = default;
31     //!< Copy assignment
32     status_code &operator=(const status_code &) = delete;
33     //!< Move assignment
34     status_code &operator=(status_code &&) = default;
35     ~status_code();
36
37     //!< Return a copy of the erased code by asking the domain to perform the erased copy.
38     status_code clone() const
39
40     //!< Implicit copy construction from any other status code if its value type is trivially copyable

```

```

41  //! and it would fit into our storage
42  template <class DomainType>
43  requires(std::is_trivially_copyable<typename DomainType::value_type>::value
44           && detail::type_erasure_is_safe<value_type, typename DomainType::value_type>::value)
45  constexpr status_code(const status_code<DomainType> &v) noexcept;
46
47  //! Implicit move construction from any other status code if its value type is trivially copyable
48  //! or move relocating and it would fit into our storage
49  template <class DomainType>
50  requires(detail::type_erasure_is_safe<value_type, typename DomainType::value_type>::value)
51  constexpr status_code(status_code<DomainType> &&v) noexcept;
52
53  //! Implicit construction from any type where an ADL discovered 'make_status_code(T, Args ...)'
54  //! returns a 'status_code'.
55  template <class T, class... Args,
56           class MakeStatusCodeOutType = decltype(make_status_code(std::declval<T>(), std::declval<
57           Args>()...))> // ADL enable
58  requires(!std::is_same<typename std::decay<T>::type, status_code>::value // not copy/move of self
59           && !std::is_same<typename std::decay<T>::type, value_type>::value // not copy/move of value
60           type
61           && is_status_code<MakeStatusCodeOutType>::value // ADL makes a status code
62           && std::is_constructible<status_code, MakeStatusCodeOutType>::value // ADLed status code is
63           compatible
64           )
65  constexpr status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(std::declval<T>(),
66  std::declval<Args>()...)));
67
68  };
69
70  //! True if the status code's are semantically equal via 'equivalent()'.
71  template <class DomainType1, class DomainType2> inline bool operator==(const status_code<DomainType1>
72  &a, const status_code<DomainType2> &b) noexcept;
73
74  //! True if the status code's are not semantically equal via 'equivalent()'.
75  template <class DomainType1, class DomainType2> inline bool operator!=(const status_code<DomainType1>
76  &a, const status_code<DomainType2> &b) noexcept;
77
78  //! True if the status code's are semantically equal via 'equivalent()' to the generic code.
79  template <class DomainType1> inline bool operator==(const status_code<DomainType1> &a, errc b)
80  noexcept;
81
82  //! True if the status code's are not semantically equal via 'equivalent()' to the generic code.
83  template <class DomainType1> inline bool operator!=(const status_code<DomainType1> &a, errc b)
84  noexcept;
85
86  //! True if the status code's are semantically equal via 'equivalent()' to the generic code.
87  template <class DomainType1> inline bool operator==(errc a, const status_code<DomainType1> &b)
88  noexcept;
89
90  //! True if the status code's are not semantically equal via 'equivalent()' to the generic code.
91  template <class DomainType1> inline bool operator!=(errc a, const status_code<DomainType1> &b)
92  noexcept;

```

3.7 Exception types

```

1  /*! Exception type representing a thrown status_code
2  */
3  template <class DomainType> class status_error;
4
5  /*! The erased type edition of status_error.

```

```

6  */
7  template <> class status_error<void> : public std::exception
8  {
9  protected:
10     /// Constructs an instance. Not publicly available.
11     status_error() = default;
12     /// Copy constructor. Not publicly available
13     status_error(const status_error &) = default;
14     /// Move constructor. Not publicly available
15     status_error(status_error &&) = default;
16     /// Copy assignment. Not publicly available
17     status_error &operator=(const status_error &) = default;
18     /// Move assignment. Not publicly available
19     status_error &operator=(status_error &&) = default;
20     /// Destructor. Not publicly available.
21     ~status_error() override = default;
22
23 public:
24     /// The type of the status domain
25     using domain_type = void;
26     /// The type of the status code
27     using status_code_type = status_code<void>;
28 };
29
30 /*! Exception type representing a thrown status_code
31 */
32 template <class DomainType> class status_error : public status_error<void>
33 {
34     status_code<DomainType> _code;
35     typename DomainType::string_ref _msgref;
36
37 public:
38     /// The type of the status domain
39     using domain_type = DomainType;
40     /// The type of the status code
41     using status_code_type = status_code<DomainType>;
42
43     /// Constructs an instance
44     explicit status_error(status_code<DomainType> code);
45
46     /// Return an explanatory string
47     virtual const char *what() const noexcept override;
48
49     /// Returns a reference to the code
50     const status_code_type &code() const &;
51     /// Returns a reference to the code
52     status_code_type &code() &;
53     /// Returns a reference to the code
54     const status_code_type &&code() const &&;
55     /// Returns a reference to the code
56     status_code_type &&code() &&;
57 };

```

3.8 Generic error coding

```
1  //!< The generic error coding (POSIX)
2  enum class errc : int
3  {
4      success = 0,      //!< This is new over std::errc
5      unknown = -1,     //!< This is new over std::errc
6
7      address_family_not_supported = EAFNOSUPPORT,
8      address_in_use = EADDRINUSE,
9      address_not_available = EADDRNOTAVAIL,
10     already_connected = EISCONN,
11     argument_list_too_long = E2BIG,
12     argument_out_of_domain = EDOM,
13     bad_address = EFAULT,
14     bad_file_descriptor = EBADF,
15     bad_message = EBADMSG,
16     broken_pipe = EPIPE,
17     connection_aborted = ECONNABORTED,
18     connection_already_in_progress = EALREADY,
19     connection_refused = ECONNREFUSED,
20     connection_reset = ECONNRESET,
21     cross_device_link = EXDEV,
22     destination_address_required = EDESTADDRREQ,
23     device_or_resource_busy = EBUSY,
24     directory_not_empty = ENOTEMPTY,
25     executable_format_error = ENOEXEC,
26     file_exists = EEXIST,
27     file_too_large = EFBIG,
28     filename_too_long = ENAMETOOLONG,
29     function_not_supported = ENOSYS,
30     host_unreachable = EHOSTUNREACH,
31     identifier_removed = EIDRM,
32     illegal_byte_sequence = EILSEQ,
33     inappropriate_io_control_operation = ENOTTY,
34     interrupted = EINTR,
35     invalid_argument = EINVAL,
36     invalid_seek = ESPIPE,
37     io_error = EIO,
38     is_a_directory = EISDIR,
39     message_size = EMSGSIZE,
40     network_down = ENETDOWN,
41     network_reset = ENETRESET,
42     network_unreachable = ENETUNREACH,
43     no_buffer_space = ENOBUFS,
44     no_child_process = ECHILD,
45     no_link = ENOLINK,
46     no_lock_available = ENOLCK,
47     no_message = ENOMSG,
48     no_protocol_option = ENOPROTOOPT,
49     no_space_on_device = ENOSPC,
50     no_stream_resources = ENOSR,
51     no_such_device_or_address = ENXIO,
52     no_such_device = ENODEV,
53     no_such_file_or_directory = ENOENT,
54     no_such_process = ESRCH,
```



```

55 not_a_directory = ENOTDIR,
56 not_a_socket = ENOTSOCK,
57 not_a_stream = ENOSTR,
58 not_connected = ENOTCONN,
59 not_enough_memory = ENOMEM,
60 not_supported = ENOTSUP,
61 operation_cancelled = ECANCELED,
62 operation_in_progress = EINPROGRESS,
63 operation_not_permitted = EPERM,
64 operation_not_supported = EOPNOTSUPP,
65 operation_would_block = EWOULDBLOCK,
66 owner_dead = EOWNERDEAD,
67 permission_denied = EACCES,
68 protocol_error = EPROTO,
69 protocol_not_supported = EPROTONOSUPPORT,
70 read_only_file_system = EROFS,
71 resource_deadlock_would_occur = EDEADLK,
72 resource_unavailable_try_again = EAGAIN,
73 result_out_of_range = ERANGE,
74 state_not_recoverable = ENOTRECOVERABLE,
75 stream_timeout = ETIME,
76 text_file_busy = ETXTBSY,
77 timed_out = ETIMEDOUT,
78 too_many_files_open_in_system = ENFILE,
79 too_many_files_open = EMFILE,
80 too_many_links = EMLINK,
81 too_many_symbolic_link_levels = ELOOP,
82 value_too_large = EOVERFLOW,
83 wrong_protocol_type = EPROTOTYPE
84 };
85
86 //! A specialisation of 'status_error' for the generic code domain.
87 using generic_error = status_error<generic_code_domain>;
88 //! A constexpr source variable for the generic code domain, which is that of 'errc'
89 //! (POSIX). Returned by 'generic_code_domain::get()'.
90 constexpr _generic_code_domain generic_code_domain;
91 // Enable implicit construction of generic_code from errc
92 constexpr inline generic_code make_status_code(errc c) noexcept;

```

3.9 errored_status_code<DomainType>

```

1 /*! A 'status_code' which is always a failure. The closest equivalent to
2 'std::error_code', except it cannot be modified, and is templated.
3
4 Differences from 'status_code':
5
6 - Never successful (this contract is checked on construction, if fails then it
7 terminates the process).
8 - Is immutable.
9 */
10 template <class DomainType> class errored_status_code : public status_code<DomainType>
11 {
12     using _base = status_code<DomainType>;
13     using _base::clear;

```

```

14     using _base::success;
15
16 public:
17     /// The type of the errored error code.
18     using typename _base::value_type;
19     /// The type of a reference to a message string.
20     using typename _base::string_ref;
21
22     /// Default constructor.
23     errored_status_code() = default;
24     /// Copy constructor.
25     errored_status_code(const errored_status_code &) = default;
26     /// Move constructor.
27     errored_status_code(errored_status_code &&) = default;
28     /// Copy assignment.
29     errored_status_code &operator=(const errored_status_code &) = default;
30     /// Move assignment.
31     errored_status_code &operator=(errored_status_code &&) = default;
32     ~errored_status_code() = default;
33
34     /// Explicitly construct from any similar status code
35     constexpr explicit errored_status_code(const _base &o) noexcept(std::is_nothrow_copy_constructible<
        _base::value>)[[expects: o.failure() == true]];
36     /// Explicitly construct from any similar status code
37     constexpr explicit errored_status_code(_base &o) noexcept(std::is_nothrow_move_constructible<_base
        >::value)[[expects: o.failure() == true]];
38
39     /// Implicit construction from any type where an ADL discovered
40     /// 'make_status_code(T, Args ...)' returns a 'status_code'.
41     template <class T, class... Args,
42             class MakeStatusCodeOutType = decltype(make_status_code(std::declval<T>(), std::declval<
        Args>()...))> // ADL enable
43     requires(!std::is_same<typename std::decay<T>::type, errored_status_code>::value // not copy/move of
        self
44             && is_status_code<MakeStatusCodeOutType>::value // ADL makes a status code
45             && std::is_constructible<errored_status_code, MakeStatusCodeOutType>::value // ADLed status
        code is compatible
46     )
47     constexpr errored_status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(std::
        declval<T>(), std::declval<Args>()...))) [[expects: make_status_code(std::forward<T>(v) /*
        unsafe? */ , std::forward<Args>(args)...).failure() == true]];
48
49     /// Explicit in-place construction.
50     template <class... Args>
51     constexpr explicit errored_status_code(in_place_t /*unused */, Args &&... args) noexcept(std::
        is_nothrow_constructible<value_type, Args &&...>::value) [[expects: _base(std::forward<Args>(
        args)... /* unsafe? */).failure() == true]];
52
53     /// Explicit in-place construction from initialiser list.
54     template <class T, class... Args>
55     constexpr explicit errored_status_code(in_place_t /*unused */, std::initializer_list<T> il, Args
        &&... args) noexcept(std::is_nothrow_constructible<value_type, std::initializer_list<T>, Args
        &&...>::value) [[expects: _base(il, std::forward<Args>(args)... /* unsafe? */).failure() ==
        true]];
56
57     /// Explicit copy construction from a 'value_type'.

```

```

58     constexpr explicit errored_status_code(const value_type &v) noexcept(std::
        is_nothrow_copy_constructible<value_type>::value) [[expects: _base(v).failure() == true]];
59
60     ///! Explicit move construction from a 'value_type'.
61     constexpr explicit errored_status_code(value_type &&v) noexcept(std::is_nothrow_move_constructible<
        value_type>::value) [[expects: _base(std::move(v) /* unsafe? */.failure() == true]];
62
63     ///! Explicit construction from an erased status code. Available only if
64     'value_type' is trivially destructible and 'sizeof(status_code) <= sizeof(status_code<erased<>>)'.
65     Does not check if domains are equal.
66     */
67     template <class ErasedType>
68     requires(detail::type_erasure_is_safe<ErasedType, value_type>::value)
69     constexpr explicit errored_status_code(const status_code<erased<ErasedType>> &v) noexcept(std::
        is_nothrow_copy_constructible<value_type>::value) [[expects: v.failure() == true]];
70
71     ///! Return a const reference to the 'value_type'.
72     constexpr const value_type &value() const &noexcept;
73 };
74
75     ///! True if the status code's are semantically equal via 'equivalent()'.
76     template <class DomainType1, class DomainType2> inline bool operator==(const errored_status_code<
        DomainType1> &a, const errored_status_code<DomainType2> &b) noexcept;
77     ///! True if the status code's are semantically equal via 'equivalent()'.
78     template <class DomainType1, class DomainType2> inline bool operator==(const status_code<DomainType1>
        &a, const errored_status_code<DomainType2> &b) noexcept;
79     ///! True if the status code's are semantically equal via 'equivalent()'.
80     template <class DomainType1, class DomainType2> inline bool operator==(const errored_status_code<
        DomainType1> &a, const status_code<DomainType2> &b) noexcept;
81     ///! True if the status code's are not semantically equal via 'equivalent()'.
82     template <class DomainType1, class DomainType2> inline bool operator!=(const errored_status_code<
        DomainType1> &a, const errored_status_code<DomainType2> &b) noexcept;
83     ///! True if the status code's are not semantically equal via 'equivalent()'.
84     template <class DomainType1, class DomainType2> inline bool operator!=(const status_code<DomainType1>
        &a, const errored_status_code<DomainType2> &b) noexcept;
85     ///! True if the status code's are not semantically equal via 'equivalent()'.
86     template <class DomainType1, class DomainType2> inline bool operator!=(const errored_status_code<
        DomainType1> &a, const status_code<DomainType2> &b) noexcept;
87     ///! True if the status code's are semantically equal via 'equivalent()' to the generic code.
88     template <class DomainType1> inline bool operator==(const errored_status_code<DomainType1> &a, errc b)
        noexcept;
89     ///! True if the status code's are semantically equal via 'equivalent()' to the generic code.
90     template <class DomainType1> inline bool operator==(errc a, const errored_status_code<DomainType1> &b)
        noexcept;
91     ///! True if the status code's are not semantically equal via 'equivalent()' to the generic code.
92     template <class DomainType1> inline bool operator!=(const errored_status_code<DomainType1> &a, errc b)
        noexcept;
93     ///! True if the status code's are not semantically equal via 'equivalent()' to the generic code.
94     template <class DomainType1> inline bool operator!=(errc a, const errored_status_code<DomainType1> &b)
        noexcept;

```

3.10 errored_status_code<erased<TRIVIALY_COPYABLE_OR_MOVE_RELOCATING_TYPE>>

```

1  template <class ErasedType> class errored_status_code<erased<ErasedType>> : public status_code<erased<
    ErasedType>>
2  {
3      using _base = status_code<erased<ErasedType>>;
4  public:
5      using value_type = typename _base::value_type;
6      using string_ref = typename _base::string_ref;
7
8      ///! Default construction to empty
9      errored_status_code() = default;
10     ///! Copy constructor
11     errored_status_code(const errored_status_code &) = default;
12     ///! Move constructor
13     errored_status_code(errored_status_code &&) = default;
14     ///! Copy assignment
15     errored_status_code &operator=(const errored_status_code &) = default;
16     ///! Move assignment
17     errored_status_code &operator=(errored_status_code &&) = default;
18     ~errored_status_code() = default;
19
20     ///! Explicitly construct from any similarly erased status code
21     constexpr explicit errored_status_code(const _base &o) noexcept(std::is_nothrow_copy_constructible<
        _base>::value) [[expects: o.failure() == true]];
22     ///! Explicitly construct from any similarly erased status code
23     constexpr explicit errored_status_code(_base &&o) noexcept(std::is_nothrow_move_constructible<_base
        >::value) [[expects: o.failure() == true]];
24
25     ///! Implicit copy construction from any other status code if its value type is trivially copyable
26     ///! and it would fit into our storage
27     template <class DomainType>
28     requires(detail::type_erasure_is_safe<value_type, typename DomainType::value_type>::value)
29     constexpr errored_status_code(const errored_status_code<DomainType> &v) noexcept [[expects: v.
        failure() == true]];
30
31     ///! Implicit construction from any type where an ADL discovered 'make_status_code(T, Args ...)'
32     ///! returns a 'status_code'.
33     template <class T, class... Args,
34             class MakeStatusCodeOutType = decltype(make_status_code(std::declval<T>(), std::declval<
                Args>()...))> // ADL enable
35     requires(!std::is_same<typename std::decay<T>::type, errored_status_code>::value // not copy/move of
        self
36             && !std::is_same<typename std::decay<T>::type, value_type>::value // not copy/move of value
            type
37             && is_status_code<MakeStatusCodeOutType>::value // ADL makes a status code
38             && std::is_constructible<errored_status_code, MakeStatusCodeOutType>::value // ADLed status
            code is compatible
39     )
40     constexpr errored_status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(std::
        declval<T>(), std::declval<Args>()...))) [[expects: make_status_code(std::forward<T>(v) /*
        unsafe? */ , std::forward<Args>(args)...).failure() == true]];
41
42     ///! Return the erased 'value_type' by value.
43     constexpr value_type value() const noexcept;
44 };

```

3.11 OS specific codes, and erased system code

```
1  //!< A POSIX error code, those returned by 'errno'.
2  using posix_code = status_code<posix_code_domain>;
3  //!< A specialisation of 'status_error' for the POSIX error code domain.
4  using posix_error = status_error<posix_code_domain>;
5
6  //!< A wrapper of 'std::error_code'.
7  using std_error_code = status_code<error_code_domain<std::error_code, detail::make_std_categories>>;
8
9  //!< A getaddrinfo error code, those returned by 'getaddrinfo()'.
10 using getaddrinfo_code = status_code<getaddrinfo_code_domain>;
11 //!< A specialisation of 'status_error' for the getaddrinfo code domain.
12 using getaddrinfo_error = status_error<getaddrinfo_code_domain>;
13
14 #ifdef _WIN32
15 //!< (Windows only) A Win32 error code, those returned by 'GetLastError()'.
16 using win32_code = status_code<win32_code_domain>;
17 //!< (Windows only) A specialisation of 'status_error' for the Win32 error code domain.
18 using win32_error = status_error<win32_code_domain>;
19
20 //!< (Windows only) A NT error code, those returned by NT kernel functions.
21 using nt_code = status_code<nt_code_domain>;
22 //!< (Windows only) A specialisation of 'status_error' for the NT error code domain.
23 using nt_error = status_error<nt_code_domain>;
24
25 /*! (Windows only) A COM error code. Note semantic equivalence testing is only
26 implemented for 'FACILITY_WIN32' and 'FACILITY_NT_BIT'. As you can see at
27 [https://blogs.msdn.microsoft.com/eldar/2007/04/03/a-lot-of-hresult-codes/](https://blogs.msdn.
    microsoft.com/eldar/2007/04/03/a-lot-of-hresult-codes/),
28 there are an awful lot of COM error codes, and keeping mapping tables for all of
29 them would be impractical (for the Win32 and NT facilities, we actually reuse the
30 mapping tables in 'win32_code' and 'nt_code'). You can, of course, inherit your
31 own COM code domain from this one and override the '_equivalent()' function
32 to add semantic equivalence testing for whichever extra COM codes that your
33 application specifically needs.
34 */
35 using com_code = status_code<com_code_domain>;
36 //!< (Windows only) A specialisation of 'status_error' for the COM error code domain.
37 using com_error = status_error<com_code_domain>;
38 #endif // _WIN32
39
40 /*! An erased-mutable status code suitably large for all the system codes
41 which can be returned on this system.
42
43 For Windows, these might be:
44
45     - 'com_code' ('HRESULT') [you need to include "com_code.hpp" explicitly for this]
46     - 'nt_code' ('LONG')
47     - 'win32_code' ('DWORD')
48
49 For POSIX, 'posix_code' and 'getaddrinfo_code' is possible.
50
51 You are guaranteed that 'system_code' can be transported by the compiler
52 in exactly two CPU registers.
53 */
```

```

54 using system_code = status_code<erased<intptr_t>>;
55
56 /*! A utility function which returns the closest matching system_code to a supplied
57 exception ptr.
58 */
59 inline system_code system_code_from_exception(std::exception_ptr &&ep = std::current_exception(),
        system_code not_matched = generic_code(errc::resource_unavailable_try_again)) noexcept;

```

3.12 Proposed `std::error` object

```

1  /*! An erased 'system_code' which is always a failure. The closest equivalent to
2  'std::error_code', except it cannot be null and cannot be modified.
3
4  This refines 'system_code' into an 'error' object meeting the requirements of
5  [P0709 Zero-overhead deterministic exceptions](https://wg21.link/P0709).
6
7  Differences from 'system_code':
8
9  - Always a failure (this is checked at construction, and if not the case,
10 the program is terminated as this is a logic error)
11 - Is immutable.
12
13 As with 'system_code', it remains guaranteed to be two CPU registers in size,
14 and trivially copyable.
15 */
16 using error = errored_status_code<erased<system_code::value_type>>;

```

3.13 `iostream` printing support

```

1  /*! Print the status code to a 'std::ostream &'.
2  Requires that 'DomainType::value_type' implements an 'operator<<' overload for 'std::ostream'.
3  */
4  template <class DomainType>
5  requires(std::is_same<std::ostream, typename std::decay<decltype(std::declval<std::ostream>()) << std::
        declval<typename status_code<DomainType>::value_type>())>::type>::value)
6  inline std::ostream &operator<<(std::ostream &s, const status_code<DomainType> &v);
7
8  /*! Print the erased status code to a 'std::ostream &'.
9  */
10 template <class ErasedType> inline std::ostream &operator<<(std::ostream &s, const status_code<erased<
        ErasedType>> &v);
11
12 /*! Print the generic code to a 'std::ostream &'.
13 */
14 inline std::ostream &operator<<(std::ostream &s, const generic_code &v);

```

3.14 `status code ptr`

```

1  /*! Make an erased status code which indirects to a dynamically allocated status code.
2  This is useful for shoeorning a rich status code with large value type into a small
3  erased status code like 'system_code', with which the status code generated by this
4  function is compatible. Note that this function can throw due to 'bad_alloc'.
5  */
6  template <class T>
7  requires(is_status_code<T>::value)
8  inline status_code<erased<typename std::add_pointer<typename std::decay<T>::type>::type>>
    make_status_code_ptr(T &&v);

```

4 Design decisions, guidelines and rationale

4.1 Do not cause `#include <string>`

`<system_error>`, on all the major STL implementations, includes `<string>` as `std::error_code::message()`, amongst other facilities, returns a `std::string`. `std::string`, in turn, drags in the STL allocator machinery and a fair few algorithms and other headers.

Bringing in so much extra stuff is a showstopper for the use of `std::error_code` in the global APIs of very large C++ code bases due to the effects on build and link times. As much as C++ Modules may, or may not, fix this some day, adopting `std::error_code` – which is highly desirable to large C++ code bases which globally disable C++ exceptions such as games – is made impossible. Said users end up having to locally reinvent a near clone of `std::error_code`, but one which doesn't use `std::string`, which is unfortunate.

Moreover, because `<stdexcept>` must include `<system_error>`, and many otherwise very simple STL facilities such as `<array>`, `<complex>`, `<iterator>` or `<optional>` must include `<stdexcept>`, we end up dragging in `<string>` and the STL allocator machinery when including those otherwise simple and lightweight STL headers for no good purpose other than that `std::error_code::message()` returns a `std::string`! That deprives very large C++ code bases of being able to use `std::optional<T>` and other such vocabulary types in their global headers.

Hence, this implicit dependency of `<system_error>` on `<string>` contravenes [P0939]'s admonition *'Note that the cost of compilation is among the loudest reasonable complaints about C++ from its users'*

It also breaks the request *'make C++ easier to use and more effective for large and small embedded systems'* by making a swathe of C++ library headers not [P0829] *Freestanding C++* compatible.

It is trivially easy to fix: stop using `std::string` to return textual representation of codes. This proposed design uses a `string_ref` instead, this is a potentially reference counted handle to a string. It is extremely lightweight, freestanding C++ compatible, and drags in no unnecessary headers.

4.2 All constexpr sourcing, construction and destruction

[*Note: <system_error> may gain the same via [P1195] Making <system_error> constexpr – end note*]

<system_error> was designed before constexpr entered the language, and many operations which ought to be constexpr for such a simple and low-level facility are not. Simple things like the `std::error_code` constructor is not constexpr, bigger things like `std::error_category` are not constexpr, and far more importantly the global source of error code categories is not constexpr, forcing the compiler to emit a magic static initialisation fence, which introduces significant added code bloat as magic fences cannot be elided by the optimiser.

The proposed replacement makes everything which can be constexpr be just that. If it cannot be constexpr, it is literal or trivial to the maximum extent possible. Empirical testing in real world code bases has found excellent effects on the density of assembler generated, with recent GCCs and clangs, almost all of the time the code generated with the replacement design is as optimal as a human assembler writer might write.

4.3 Header only libraries can now safely define custom code categories

[*Note: <system_error> may gain the same via [P1196] Value-based std::error_category comparison – end note*]

Something probably unanticipated at the time of the design of <system_error> is that bespoke `std::error_category` implementations are unsafe in header only libraries. This has caused significant, and usually unpleasant, surprise in the C++ user base.

The problem stems from the comparison of `std::error_category` implementations which is *required* by the C++ standard to be a comparison of address of instance. When comparing an error code to an error condition, the `std::error_category::equivalent()` implementation compares the input error code's category against a list of error code categories known to it in order to decide upon equivalence. This is by address of instance.

Header only libraries must use Meyer singletons to implement the source of the custom `std::error_category` implementation i.e.

```
1 inline const my_custom_error_category &custom_category()  
2 {  
3     static my_custom_error_category v;  
4     return v;  
5 }
```

Ordinarily speaking, the linker would choose one of these inline function implementations, and thus `my_custom_error_category` gets exactly one instance, and thus one address in the final executable. All would therefore seem good.

Problems begin when a user uses the header only library inside a shared library. Now there is a single instance of the inline function *per shared library*, not per final executable. It is not uncommon for users to use more than one shared library, and thus multiple instances of the inline function

come into existence. You now get the unpleasant situation where there are multiple singletons in the process, each with a different address, despite being the same error code category. Comparisons between error codes and categories thus subtly break in a somewhat chance based, hard to debug, way¹.

Those bitten by this ‘feature’ tend to be quite bitter about it. This author is one of those embittered. He has met others who have been similarly bitten through the use of ASIO and the Boost C++ Libraries. It’s a niche problem, but one which consumes many days of very frustrating debugging for the uninitiated.

The proposed design makes error category sources all-constexpr as well as error code construction. This is incompatible with singletons, so the proposed design does away with the need for singleton sources entirely in favour of stateless code domains with a static random unique 64-bit id, of which there can be arbitrarily many instantiated at once, and thus the proposed design is safe for use in header only libraries.

In case there is concern of collision in a totally random unique 64 bit id, here are the number of random 64-bit numbers needed in the same process space for various probabilities of collision (note that 10e15 is the number of bits which a hard drive guarantees to return without mistake):

Probability of collision	10e-15	10e-12	10e-9	10e-6	10e-3 (0.1%)	10e-2 (1%)
Random 64-bit numbers needed	190	6100	190,000	6,100,000	190,000,000	610,000,000

4.4 No more `if(!ec)` ...

[*Note: <system_error> may gain the same via [P1198](#) `Adding error_category::failed()`*
– end note]

`std::error_code` provides a boolean test. The correct definition for the meaning of the boolean test is ‘is the value in this error code all bits zero, ignoring the category?’. It does **not** mean ‘is there no error?’.

This may seem like an anodyne distinction, but it causes real confusion. During a discussion on the Boost C++ Libraries list regarding this issue, multiple opinions emerged over whether this was ambiguous, whether it would result in bugs, whether it was serious, whether programmers who wrote the code assuming the latter were the ones at fault, or whether it was the meaning of the boolean test. No resolution was found.

All this suggests to SG14 that there is unhelpful ambiguity which we believe can never lead to better quality software, so we have removed the boolean test in the proposed design. Developers must now be clear as to exactly what they mean: `if(ec.success()) ...`, `if(ec.failure()) ...` and so on.

¹Do inline variables help? Unfortunately not. They suffer from the same problem of instance duplication when used in shared libraries. This is because standard C++ code has no awareness of shared libraries.

4.5 No more filtering codes returned by system APIs

Because `std::error_code` treats all bits zero values specially, and its boolean test does not consider category at all, when constructing error codes after a syscall, one must inevitably add some logic which performs a local check of whether the system returned code is a failure or not, and only then follow the error path.

This is fine for a lot of use cases, but many platforms, and indeed third party libraries, like to return success-with-information or success-with-warning codes. The current `<system_error>` does not address the possibility of multiple success codes being possible, nor that there is any success code other than all bits zero.

It also forces the program code which constructs the system code into an error code to be aware of implementation details of the source of the code in order to decide whether it is a failure or not. That is usually the case, but is not always the case. For where it is not the case, forcing this on users breaks clean encapsulation.

The proposed redesign accepts unfiltered and unmodified codes from any source. The category – called a *domain* in this proposal – interprets codes of any form of success or failure. Users can always safely construct a `status_code` (in this proposal, not [P0262]’s `status_value`) without knowing anything about the implementation details of its source. No one value is treated specially from any other.

4.6 All comparisons between codes are now semantic, not literal

Even some members of WG21 get the distinction between `std::error_code` and `std::error_condition` incorrect. That is because they appear to be almost the same thing, the same design, same categories, with only a vague documentation that one is to be used for system-specific codes and the other for non-system-specific codes.

This leads to an unnecessarily steep learning curve for the uninitiated, confusion amongst programmers reading code, incorrect choice of `std::error_condition` when `std::error_code` was meant, surprise when comparisons between codes and conditions are semantic not literal, and more of that general ambiguity and confusion we mentioned earlier.

The simple solution is to do away with all literal comparison entirely. Comparisons of `status_code` are **always** semantic. If the user really does want a literal comparison, they can manually compare domain and values by hand. Almost all of the time they actually want semantic comparison, and thus `operator ==`’s non-regular semantic comparison is exactly right.

4.7 `std::error_condition` is removed entirely

As comparisons are now always semantic between `status_code`’s, there is no longer any need for a distinction between `std::error_code` and `std::error_condition`. We therefore simplify the situation by removing any notion of `std::error_condition` altogether.

4.8 `status_code`'s value type is set by its domain

`std::error_code` hard codes its value to an `int`, which is problematic for third party error coding schemes which use a `long`, or even an `unsigned int`. `status_code<DomainType>` sets its `value_type` to be `DomainType::value_type`. Thus if you define your own domain type, its value type can be any type you like, including a structure or class.

This enables *payload* to be transmitted with your status code e.g. if the status code represents a failure in the filesystem, the payload might contain the path of a relevant file. It might contain the stack backtrace of where a failure or warning occurred, a `std::exception_ptr` instance, or anything else you might like.

We make great use of this domain definable value type facility to wrap up all possible `std::error_code`'s into status codes via a code domain whose value type is a `std::error_code`. This enables complete participation of any existing error code scheme within the proposed status code scheme.

4.9 `status_code<DomainType>` is type erasable

`status_code<DomainType>` can be type erased into a `status_code<void>` which is an immutable, unrelatable, uncopyable type suitable for passing around by const lvalue reference only. This allows non-templated code to work with arbitrary, unknown, `status_code<DomainType>` instances. One may no longer retrieve their value obviously, but one can still query them for whether they represent success or failure, or for a textual message representing their value, and so on.

If, and only if, `DomainType::value_type` and some type `U` are `TriviallyCopyable` and the size of `DomainType::value_type` is less than or equal to size of `U`, an additional type erasure facility becomes available, that of `status_code<erased<U>>`. Unlike `status_code<void>`, this type erased form is copyable which is safe as `DomainType::value_type` and `U` are `TriviallyCopyable`, and are therefore both copyable as if via `memcpy()`.

This latter form of type erasure is particularly powerful. It allows one to define some global `status_code<erased<U>>` which is common to all code: `status_code<erased<intptr_t>>` would be a very portable choice². Individual components may work in terms of `status_code<LocalErrorType>`, but all public facing APIs may return only the global `status_code<erased<intptr_t>>`. This facility thus allows any arbitrary `LocalErrorType` to be returned, unmodified, *with value semantics* through code which has no awareness of it. The only conditions are that `LocalErrorType` is trivially copyable, and is not bigger than the erased `intptr_t` type.

4.10 More than one 'system' error coding domain: `system_code`

`std::system_category` assumes that there is only one 'system' error coding, something mostly true on POSIX (though note that POSIX's error coding is always a subset of the POSIX implementation's error coding), but not elsewhere, especially on Microsoft Windows where at least four primary system

²Why? On x64 with SysV calling convention, a trivially copyable object no more than two CPU registers of size will be returned from functions via CPU registers, saving quite a few CPU cycles. AArch64 will return trivially copyable objects of up to 64 bytes via CPU registers!

error coding schemes exist: (i) POSIX `errno` (ii) Win32 `GetLastError()` (iii) NT kernel `NTSTATUS` (iv) COM/WinRT/DirectX `HRESULT`.

The proposed library makes use of the `status_code<erased<U>>` facility described in the previous section to define a type alias `system_code` to a type erased status code sufficiently large enough to carry any of the system error codings on the current platform. This allows code to use the precise error code domain for the system failure in question, and to return it type erased in a form perfectly usable by external code, which need neither know nor care that the failure stemmed originally from COM, or Win32, or POSIX. All that matters is that the status code semantically compares true to say `std::errc::no_such_file_or_directory`.

4.11 `std::errc` gets its own code domain `generic_code`, eliminating `std::error_condition`

Similar, but orthogonal, to `system_code` is `generic_code` which has a value type of the strongly typed enum `std::errc`. Codes in the generic code domain become the ‘portable error codes’ formerly represented by `std::error_condition` in that they act as semantic comparator of last resort.

Generic codes allow one to write code which semantically compares success or failure to the standard failure reasons defined by POSIX. This allows one to write portable code which works independent of platform and implementation.

5 Technical specifications

No Technical Specifications are involved in this proposal.

6 Frequently asked questions

6.1 Implied in this design is that code domains must do nothing in their constructor and destructors, as multiple instances are permitted and both must be trivial and constexpr. How then can dynamic per-domain initialisation be performed e.g. setting up at run time a table of localised message strings?

The simplest is to use statically initialised local variables, though be aware that it is always legal to use status code from within static initialisation and finalisation, so you need to lazily construct any tables on first use and never deallocate. Slightly more complex is to use the domain’s `string_ref` instances to keep a reference count of the use of the code domain, when all `string_ref` instances are destroyed, it is safe to deallocate any per-domain data.

7 Acknowledgements

Thanks to Ben Craig, Arthur O'Dwyer and Vicente J. Botet Escriba for their comments and feedback.

Thanks to Jesse Towner for walking through the code and pointing out lots of small mistakes, as well as contributing a patchset fixing memory leaks, improving constexpr type erasure, and other implementation improvements.

8 References

- [N2066] Beman Dawes,
TR2 Diagnostics Enhancements
<https://wg21.link/N2066>
- [P0262] Lawrence Crowl, Chris Mysen,
A Class for Status and Optional Value
<https://wg21.link/P0262>
- [P0709] Herb Sutter,
Zero-overhead deterministic exceptions: Throwing values
<https://wg21.link/P0709>
- [P0824] O'Dwyer, Bay, Holmes, Wong, Douglas,
Summary of SG14 discussion on <system_error>
<https://wg21.link/P0824>
- [P0829] Ben Craig,
Freestanding proposal
<https://wg21.link/P0829>
- [P0939] B. Dawes, H. Hinnant, B. Stroustrup, D. Vandevorode, M. Wong,
Direction for ISO C++
<http://wg21.link/P0939>
- [P1029] Douglas, Niall
move = relocates
<https://wg21.link/P1029>
- [P1031] Douglas, Niall
Low level file i/o library
<https://wg21.link/P1031>
- [P1095] Douglas, Niall
Zero overhead deterministic failure – A unified mechanism for C and C++
<https://wg21.link/P1095>

- [P1144] O'Dwyer, Arthur
Object relocation in terms of move plus destroy
<https://wg21.link/P1144>
- [P1195] Dimov, Peter
Making `<system_error>` constexpr
<https://wg21.link/P1195>
- [P1196] Dimov, Peter
Value-based `std::error_category` comparison
<https://wg21.link/P1196>
- [P1197] Dimov, Peter
A non-allocating overload of `error_category::message()`
<https://wg21.link/P1197>
- [P1198] Dimov, Peter
Adding `error_category::failed()`
<https://wg21.link/P1198>
- [P1631] Douglas, Niall and Steagall, Bob
Object detachment and attachment
<https://wg21.link/P1631>
- [1] *Boost.Outcome*
Douglas, Niall and others
<https://ned14.github.io/outcome/>
- [2] *stl-header-heft github analysis project*
Douglas, Niall
<https://github.com/ned14/stl-header-heft>